

Lab 2: Pipelined RISC-V Processor

Student ID: 314551138

10 13, 2025

1 Introduction

In this lab, we develop an iterative integer multiply/divide unit in Verilog, moving step by step from basic iterative designs to optimized architectures. The work begins with the construction of the iterative multiplier and divider, then improves efficiency through the Modified Booth algorithm, and finally extends functionality with a three-input multiplier built from Modified Booth units. In addition to building and connecting these modules, we also wrote test cases to check the results and confirm the cycle counts.

2 Design

This section describes the design and implementation of the multiplier and divider.

2.1 Iterative Multiplier

The iterative multiplier is implemented using the shift-and-add algorithm shown in Figure 3. The datapath follows the reference design in Figure 6 with registers for operands, partial results, a counter, and a sign bit, together with an adder and multiplexers. Each cycle shifts `a_reg` left, shifts `b_reg` right, and conditionally adds `a_reg` to the result if `b0` is set, completing the operation in 33 cycles. Signed operands are converted to unsigned form before computation, and the final sign is corrected with `sign_reg`. Unlike the reference datapath, the `unsign` block and `sign_mux_sel` were not implemented as separate little units, instead, both the conversion and sign correction are handled directly in the datapath. In addition, the counter is reset to zero and then loaded to 31 during initialization, rather than being preset to 31. These simplifications reduce hardware and are easier for me to implement than original one and without affecting correctness or cycle count.

2.2 Iterative Divider

The iterative divider is based on the restoring division algorithm in Figure 4, with the datapath shown in Figure 7. The design uses a 65-bit `a_reg` for the partial remainder, a 65-bit `b_reg` for the divisor, a counter, and sign registers. In each cycle, `a_reg` is shifted left and subtracted by `b_reg`; if the result is negative, the shifted value is restored, otherwise the difference is kept with the LSB set to 1. After 32 iterations, the result `{remainder||quotient}` is produced in 33 cycles. Signed division is supported by converting operands to magnitude form and later correcting the signs: the quotient sign is given by the XOR of input signs, while the remainder follows the dividend. Compared with the reference datapath, sign handling is integrated directly in the datapath and a 66-bit subtractor is used to correctly handle overflow, the reason is that it simplifies the design without changing correctness and same as the multiplier this kind of design is easier for me to implement the datapath and function.

2.3 Booth Multiplier

The Modified Booth multiplier applies the algorithm in Figure 5, processing two multiplier bits per iteration and thus reducing the cycle count to about half of the iterative multiplier. In each cycle, three bits of the multiplier ($b[2:0]$) determine the partial product as $+A$, $+2A$, $-A$, $-2A$, or zero, followed by shifting `a_reg` left by two and `b_reg` right by two. The datapath is derived from the iterative multiplier with minor changes: a Booth encoder part is added, `b_reg` is extended by one dummy bit for encoding, and the counter decrements by two per cycle until 16 iterations are completed. Sign handling is simpler than in the iterative multiplier, since the Booth encoder inherently takes care of positive and negative partial products. Therefore, we reduce most of the sign handling part in the iterative multiplier. This design preserves correctness while reducing complexity of hardware and latency from 33 cycles to around 17 cycles.

2.4 Three-Input Multiplier

In the design of the three-input iterative multiplier, I adopted a pipeline-like structure that separates the computation into three stages. In the first stage, a Booth multiplier computes the 64-bit product of inputs A and B. In the second stage, this product is divided into its high and low 32-bit parts, each of which is multiplied by the input C. To ensure correctness, the value of C is stored in an additional register when the $A \times B$ result becomes available, so that C is passed along the pipeline and always aligned with the corresponding AB result. In the third stage, the partial results $hi \times C$ and $lo \times C$ are combined, and when the low part is negative, a correction term `corr` is added to guarantee mathematical correctness. Compared to the earlier FSM-controlled multiplier, the three-input design uses this pipeline-like method, which is simpler in structure and easier to implement.

3 Testing Methodology

3.1 Multiplier (Iterative, Booth)

For the multiplier units, I tested a variety of input pairs, focusing on correctness and corner cases. The test cases included:

- **Zero operations:** $0 \times (+N) = 0$, $0 \times (-N) = 0$
To confirm that multiplication with zero always produces zero regardless of the sign.
- **One tests:** $1 \times (\text{max positive}) = \text{max positive}$, $-1 \times (\text{max positive}) = -(\text{max positive})$
To check correct sign extension when multiplying by 1 and -1.
- **Corner cases:** $-1 \times (\text{min negative})$ (overflow), $(\text{max positive}) \times (\text{max positive})$, $(\text{min negative}) \times (\text{min negative})$
To stress-test extreme values and observe overflow behavior.
- **Sign handling:** $-1 \times 2 = -2$, $-1 \times -1 = 1$
To validate correct handling of negative operands and signed boundaries.

3.2 Divider/Reminder (Iterative)

For the divider unit, I prepared test cases that cover correctness, corner cases, and different combinations of signed/unsigned operations.

- **Basic correctness:** Basic cases such as $1/1 = 1$, $0/1 = 0$, and $n/n = 1$ were tested to ensure functional behavior of signed and unsigned operation.

- **Zero divisor cases:** When dividing by zero, the result follows (quotient = -1, remainder = quotient).
- **Signed corner cases:** For example, $-2^{31}/-1 = -2^{31}$ (overflow case), and small dividend < divisor cases which should yield (quotient = dividend and remainder = 0.)
- **Unsigned division:** Large numbers divided by small numbers, e.g., $0xffffffff/1 = 0xffffffff$, or equal-division cases to verify correctness of unsigned path.
- **Interleaved signed/unsigned tests:** During debugging, we found that running only signed or only unsigned operations sequentially could hide bugs, since operand registers and sign-handling logic might be contaminated by later test cases. Therefore, we deliberately alternated signed and unsigned operations in the test sequence to validate robustness of the implementation.

3.3 Three Input Multiplier

For the three-input multiplier, we tested both correctness and corner cases with different (A, B, C) triplets. The datapath was organized into pipeline-like stages, and during debugging we identified two key issues: (1) the low 32-bit slice (`lo`) sometimes behaved like a signed value, requiring an explicit correction term; (2) the C value (`c_reg`) could be overwritten by the next test case if not protected, so we also designed tests where C alternates between positive and negative values to verify alignment.

The test cases included:

- **Basic correctness:** $1 \times 1 \times 5 = 5$, $2 \times 3 \times 2 = 12$, $2 \times 3 \times 0 = 0$.
- **lo part correction (lo slice):** $(2^{31} - 1) \times (-2^{31}) \times 2$, $(2^{31} - 1) \times (-2^{31}) \times (-1)$, ensuring that the correction term for `lo` yields the correct 96-bit result.
- **C alternation:** $2 \times 3 \times (2) = 12$, repeated after earlier $2 \times 3 \times (-2)$, to confirm `c_reg` is not polluted by subsequent case.

This methodology systematically stresses both the signed/unsigned behavior of the `lo` path and the robustness of C propagation through the pipeline.

4 Evaluation

4.1 Simulation Results

We simulated the Iterative Multiplier/Divider, the Booth Multiplier, and the Three-Input Multiplier using a variety of test cases. The results of the operations matched the expected outputs in all cases, confirming correctness. As for threeInput multiplier, I write a simulator like the three others to show the cycle counts.

Unit	Operation	Result	Cycles
Iterative Mul	$0x0badbee f \times 0x10000000$	$0x0badbee f0000000$	33
Iterative Div	$0xf5fe4fbc \div 0x00004eb6$	$0xffffdf75$	33
Iterative Rem	$0x08a22334 \bmod 0xfdcba02b$	$0x020503b5$	33
Iterative Divu	$0xf5fe4fbc \div_u 0x00004eb6$	$0x00032012$	33
Iterative Remu	$0x0a56adca \bmod_u 0xfabc1234$	$0x0a56adca$	33
Booth Mul	$0x0badbee f \times 0x10000000$	$0x0badbee f0000000$	17
Three-Input Mul	$(0x7fffffff \times 0x80000000) \times 0x00000002$	$0xffffffff8000000100000000$	36
Three-Input Mul	$(0xffffffff \times 0x00000001) \times 0xffffffff$	$0x000000000000000000000001$	36

Table 1: Simulation results and cycle counts

4.2 Correctness Testing

The test inputs included zero cases, maximum/minimum values, signed and unsigned operands, and corner cases such as $-2^{31}/-1$ (overflow). For the three-input multiplier, we also verified cases where the lower 32-bit slice of $A \times B$ became negative and required correction, as well as cases where alternating positive/negative C values ensured that the C register was not polluted by the next case. Luckily, all outputs for the testcases matched expectations.

5 Conclusion

The evaluation shows that all multiplier and divider designs produced correct results across normal and corner cases. Quantitatively, the iterative unit required 33 cycles for all operations, the Booth multiplier improved efficiency to 17 cycles, and the three-input multiplier completed in 36 cycles. Qualitatively, the Booth design demonstrated clear performance advantages, while the three-input multiplier, though structurally simple, was harder to debug due to operand propagation issues. Using GTKWave helped locate the root cause of C value pollution across test cases, and much of the effort in this lab was spent resolving such operand contamination problems.